

Mermaid.js Notation Guide for Process Modeling

Mermaid.js is a text-based diagramming language that enables business users to generate process diagrams in code. A `.mmd` definition begins with a diagram type and then lists nodes, edges, and other elements. For example, a flowchart starts with `flowchart LR`, a sequence diagram with `sequenceDiagram`, etc. Flowcharts (nodes and arrows) and other diagram types are written with a simple, Markdown-like syntax ¹. Mermaid then renders the diagram automatically.

This guide serves as a **syntax dictionary** and **best-practice handbook** for generating readable process models (e.g. order-to-cash, onboarding, incident handling, approvals) in Mermaid. It covers each diagram type, layout techniques, strengths/limits vs BPMN and PlantUML, and rules-of-thumb for clarity (branching, swimlane simulation, labeling, alignment). Examples use generic processes that apply across sectors.

TIP: Mermaid's `flowchart` keyword is synonymous with `graph` – you can write `graph LR` instead of `flowchart LR` ². Also, Mermaid supports Markdown in node/edge labels; e.g. use `**bold**` or `*italics*` and it will auto-wrap long labels ³.

Flowchart Diagrams

Flowcharts are defined by nodes (boxes, circles, etc.) and arrows (lines). A flowchart begins with:

```
flowchart LR
  A[Start] --> B{Decision}
  B -->|Yes| C[Action 1]
  B -->|No| D[Action 2]
  C --> E[End]
  D --> E
```

- **Direction:** After `flowchart`, specify orientation: `LR` (left→right), `TB` or `TD` (top→bottom), `RL`, etc. ⁴. For example, `flowchart LR` makes a left-to-right flow. - **Nodes:** Enclose text in shape delimiters: - **Rectangle:** `A[Text]` or `A["Text"]` - **Round:** `A(Rounded)` - **Diamond (decision):** `A{Decision}` - **Stadium, subroutine, cylinder, circle, etc.** Mermaid provides many shapes ⁵ (e.g., use `A(("Oval"))`, `B>Rect`, `C((Cylinder))`). - **Edges:** Use `--`, `-->`, `-.>`, `<--` with optional labels: - Unlabeled arrow: `A --> B` - With label: `A -->|yes| B` (labels go between `|`). - **Multi-directional arrows:** You can create special arrowheads: e.g. `A ---o B` (circle at end), `A ---x B` (cross), `A -.-> B` (open arrow, async) ⁶. - **Node Labels vs IDs:** The text in `[]` is shown; the ID (e.g. `A`) is the hidden reference. You can assign different ID and label (e.g. `A["Start Process"]`) to keep labels descriptive.

WARNING: The lowercase word `end` is reserved in flowcharts and will break the diagram if used as a node ID. If you need an "End" node, capitalize or quote it (e.g. `End`, `"end"`) ⁷.

WARNING: Node IDs beginning with `o` or `x` can be mistaken for special characters. Prefix

them with a space or capitalize the letter (e.g. `" dev": A--- " ops"`, or `Dev---Ops`)⁸.

Subgraphs (Lanes or Groups): Flowcharts support subgraphs to group nodes, e.g.:

```
flowchart TB
  subgraph Procurement
    A[Initiate Order] --> B[Approve Budget]
  end
  subgraph Fulfillment
    C[Receive Order] --> D[Pack Items]
  end
  A --> C %% link between subgraphs
```

Each `subgraph Name` / `end` block groups nodes. You can link in/out of a subgraph as shown. Note: if any node inside a subgraph connects outside, the subgraph inherits the parent graph's layout⁹. Use `direction` inside a subgraph to override its internal flow (as long as no external links exist).

Text Formatting and Wrapping: Mermaid supports Markdown-style text in labels: - Use `**bold**` or `*italics*`. - Long labels auto-wrap (no need for `<br>` tags) by default³.

Disable wrapping via frontmatter:

```
---
config:
  markdownAutoWrap: false
---
graph LR
  A[This is a very long label<br>with a manual break] --> B
```

Without explicit `markdownAutoWrap: false`, labels will split at natural word boundaries.

Edge Styling (optional): You can style arrows using `linkStyle` or shape curves with `curve` directives. See Mermaid docs for advanced styling (e.g. dash vs solid, stroke color).

Sequence Diagrams

Sequence diagrams model interactions over time (messaging between actors). Begin with `sequenceDiagram`:

```
sequenceDiagram
  participant User
  participant HR
  participant IT
```

```
User->>HR: Submit paperwork
HR->>IT: Set up accounts
IT-->>User: Send welcome info
```

- **Participants:** Defined automatically from usage or explicitly via `participant X`. Use `actor X` for stick-figure icons. Actors appear left-to-right in order of declaration; you can override order with an additional syntax (e.g. `participant A order 1`). - **Messages:** The format is `[Actor]->>[Actor]: Message text`. Arrows can be solid or dotted, with arrowheads or without. Mermaid supports several arrow types ¹⁰:

- `->` (solid, no arrow), `-->` (dotted, no arrow)
- `->>` (solid with arrowhead), `-->>` (dotted with arrowhead)
- Bidirectional: `<->` / `<-->` (solid/dotted) ¹⁰.
- Terminating in a cross (`-x`, `--x`) or open arrow (`-`, `--`) for async messages.

- **Activations:** You can highlight lifelines with `activate A` / `deactivate A`, or use the shorthand `A+` at the end of a message to auto-activate/deactivate. - **Notes and Loops:** Add `Note over A: text` or `Note left of A: text`. Loops (`loop ... end`) and alternatives (`alt ... else ... end`) can structure the diagram for conditional or repeating interactions. - **Grouping:** Use `box <color> Title` to group actors (with an optional background color) ¹¹. This can simulate swimlane-like grouping in a sequence context.

TIP: Enclose multi-word participant names in quotes or give them aliases if needed. For example:

```
participant Alice as "Alice A."
participant Bob as "Bob B."
Alice->>Bob: Hi
```

TIP: Use `loop` / `alt` to show repeated or conditional flows; `par` blocks can model parallel interactions.

State Diagrams

State diagrams (stateMachine) describe how a system transitions between states. Start with `stateDiagram-v2`:

```
stateDiagram-v2
    [*] --> Idle
    Idle --> Processing : start
    Processing --> Idle : done
    Processing --> [*] : fail
```

- **States:** Declare a state by name (e.g. `Idle`). You can label a state by `state Idle: Description` or `Idle "Description"`. - **Transitions:** Use `State1 --> State2` for a transition. Add a label after a colon (e.g. `Idle --> Processing : start`) to describe the event or action. If you refer to an undefined state in a transition, Mermaid will auto-create it. - **Initial/Final:** The special state `[ * ]` indicates start or

end. An arrow `[*] --> State` makes State the start; `State --> [*]` makes it terminal ¹² . -

Composite States: To show nested states, use `state S { ... }`. Inside a composite state block, list sub-states/transitions. For example:

```
stateDiagram-v2
    state Active {
        [*] --> SubState1
        SubState1 --> SubState2
        SubState2 --> [*]
    }
    Active --> Inactive : deactivate
```

- **Parallel States:** (v10.2.0+) Use `||` within a state to define parallel regions.

TIP: Mermaid's state syntax is designed to resemble PlantUML's, making it easier to migrate models ¹³ . Use clear labels on transitions to indicate events/conditions.

Class Diagrams

Mermaid can draw UML-like class diagrams with `classDiagram`. Syntax highlights:

```
classDiagram
    class Customer {
        +String name
        +String email
        +placeOrder()
    }
    class Order {
        -int id
        -float total
        +calculateTotal()
    }
    Customer "1" --o "*" Order : places
    Order "1" --> "1" Payment : pays with
```

- **Classes:** Use `class ClassName { ... }` to define members. Inside `{}`, prefix attributes with `+` (public) or `-` (private), and methods with `()`.

- **Shorthand:** You may also use `ClassName : +field` or `ClassName : method()` lines outside a block to add members. - **Relationships:** UML relations are indicated by arrows ¹⁴ . Common types: - `<|--`

Inheritance (generalization)

- `*--` Composition
- `o--` Aggregation
- `-->` Simple association
- `..>` Dependency, `..|>` Realization, etc. ¹⁴ .

For example, `Vehicle <|-- Car` makes Car inherit Vehicle. - **Multiplicity/Cardinality:** You can label

ends of associations by putting a quoted number or range before the arrow. E.g.

`Customer "1" o-- "*" Order` denotes one-to-many ¹⁵. Place the quotes on either side of the arrow as needed. - **Annotations:** Mark classes with stereotypes by using `<<Interface>>` or `<<Abstract>>`. You can attach them inline (inside the class name) or on a separate line. - **Direction:** Optionally set diagram orientation with `direction TB` or `LR` before definitions.

TIP: After drawing a class diagram, use `classDiagram direction LR` to switch layout. Keep names short and avoid overly complex class hierarchies in one diagram.

Gantt Charts

Mermaid Gantt charts visualize timelines. Begin with `gantt`:

```
gantt
  title Project Plan
  dateFormat YYYY-MM-DD
  axisFormat %Y-%m-%d
  section Phase 1
    Task A :a1, 2025-01-01, 10d
    Task B :after a1, 5d
  section Phase 2
    Task C :2025-01-15, 7d
    Milestone M: milestone, 2025-01-22
```

- **Title and Dates:** Use `title` for a chart title (optional) ¹⁶. Specify `dateFormat` (input format) and `axisFormat` (display format). - **Sections:** Group tasks with `section Name`. Each section heading appears on the left. - **Tasks:** Each task line has `TaskName :id, start, duration/end`. After the colon `:`, you can include tags (`active`, `done`, `crit`, `milestone`). Then give either: - An explicit end date (e.g. `2025-01-10`), **or** - A duration (e.g. `10d` = 10 days).

Or to set dependencies: use `after <otherID>` to start after another task's finish. - **Defaults:** Tasks are sequential by default (each starts when the previous ends) unless you use `after`. - **Exclusions:** Use `excludes` keyword (in a preamble) to skip weekends or holidays.

TIP: For clarity, label your IDs meaningfully (e.g. `a1`, `b2`) and use `after id` to create explicit dependencies. The Gantt's time axis will auto-scale to fit. Mermaid's Gantt will push tasks to the right if exclusions (like weekends) occur ¹⁷.

User Journey Diagrams

Mermaid's **journey** diagrams map user steps (common in UX/workflow analysis). Syntax:

```
journey
  title Customer Onboarding
```

```

section Account Setup
  "Enter details": 5: User
  "Verify email": 3: User, System
section First Purchase
  "Browse products": 4: User
  "Complete checkout": 2: User

```

- Start with `journey` then `title` (optional). - Divide work into `section` blocks. Each section is a stage (e.g. "Pre-arrival", "Day 1"). - **Tasks syntax:** `"Task name": <score>: <Actor1>, <Actor2>`¹⁸. The number score indicates, for example, user satisfaction or sequence order; actors list who is involved. - Mermaid will layout each section's tasks in order of their scores (highest first by default).

TIP: Journey charts emphasize user perspective. Use quotes around multi-word tasks. Scores can be used to rank tasks or indicate importance.

Pie Charts

For simple proportions, use `pie`:

```

pie showData title "Request Types"
  "Bug" : 30
  "Feature" : 55
  "Task" : 15

```

- Begin with `pie`. Optionally add `showData` (to display values on slices) and `title`. - List slices as `"Label": value`. Values must be positive numbers¹⁹. - Example above shows that 55% of requests are Features, 30% Bugs, 15% Tasks.

TIP: Keep pie labels short. Use `showData` to add numeric labels if helpful.

Layout & Readability Techniques

To **auto-generate** clear diagrams, use these layout tips:

- **Direction and Spacing:** Explicitly set direction (`LR` vs `TB`) to best fit the data flow. Horizontal (`LR`) works well for decision-heavy flows; vertical (`TB`) for linear progressions. If diagrams become large, consider splitting into subgraphs or separate charts.
- **Subgraphs as Lanes:** Mermaid doesn't have native "swimlanes", but use subgraphs with headings to simulate lanes or phases. Label the subgraph (e.g. `subgraph Finance`) to act as a lane. Remember that linking across subgraphs will force an inherited layout⁹.
- **Text Wrapping:** Enable multiline labels for nodes or edges by using newline characters within the quotes. Don't cram text on one line. Mermaid's auto-wrap (Markdown strings) helps readability³.

- **Styling Layout:** Mermaid (v11+) supports layout engines. The default **Dagre** is fine for most flows, but for very complex models you can integrate ELK (Eclipse Layout Kernel) to optimize node placement ²⁰ . In practice, try reorganizing nodes or adding dummy edges to improve spacing.
- **Avoid Overcrowding:** If a diagram is too dense, break it into smaller subprocesses or use notes/comments to declutter. Use the `%%` comment syntax (lines starting with `%%`) to leave inline notes that aren't rendered ²¹ .
- **Colors & Styles:** Use class-based styling or themes sparingly (see Mermaid theming docs) to highlight critical paths or separate steps. For example, you might color all gateway nodes or highlight exceptions in red.

TIP: Create a small layout legend (e.g. subgraph labels or color codes) for complex flows. Preview diagrams in **Mermaid Live Editor** or your chosen tool to tweak node positions. Remember that different renderers (VS Code, GitHub, Obsidian, etc.) may space elements slightly differently, so test in your target platform.

Comparative Strengths & Limitations

Mermaid fills the gap between code and diagrams. Compared to **BPMN** and **PlantUML**:

- **Mermaid vs BPMN:** BPMN (Business Process Model and Notation) is a formal, industry-standard language with rich constructs (tasks, events, gateways, pools/lanes, error handling, etc.). **Mermaid does not natively support full BPMN semantics** (there's no built-in "exclusive gateway" or "parallel gateway" icon, no pools). Mermaid's flowcharts are simpler: you must simulate gateways with diamonds and label edges manually. A BPMN-like diagram requires custom workarounds. In fact, Mermaid users have requested true BPMN support (e.g. for Notion use cases) ²² , but it's not (yet) available. The upside: Mermaid's syntax is lightweight, so it's easier for non-experts to start drawing a process flow, but at the cost of less expressive power.
- **Mermaid vs PlantUML:** Both are text-based and support diagrams, but PlantUML has a much broader feature set. PlantUML can draw almost any UML or architecture diagram (sequence, class, use-case, activity, component, etc.) plus supports skinning and DI attributes. **Mermaid's syntax is simpler and more Markdown-like**, making it quick to learn ²³ . PlantUML, in contrast, offers fine-grained control and more diagram types but has a steeper learning curve ²³ . For example, PlantUML has native swimlane charts and BPMN support (with extensive shapes), while Mermaid does not. On the other hand, Mermaid is widely supported in documentation platforms (GitHub, Notion, etc.) out-of-the-box ²⁴ ²⁵ , and is ideal for lightweight diagrams embedded in text.

NOTE: Choose Mermaid when ease of use and platform integration matter most. For strict BPMN compliance or highly detailed technical diagrams, a specialized tool (BPMN editor, PlantUML or UML tools) may be more appropriate.

Example Process Models

Below are some universal process examples written in Mermaid syntax. These illustrate how to model common workflows.

Order-to-Cash (Flowchart)

```

flowchart LR
    A[Receive Order] --> B[Process Payment]
    B --> C{Payment OK?}
    C -->|Yes| D[Pack Items]
    C -->|No| E[Notify Customer]
    D --> F[Ship Order]
    F --> G[Complete Order]

```

Description: This flowchart shows an order entering the system, payment processing, and branching on approval. Decision node **C** labels the “Yes/No” paths. Each step is a node (rectangle).

Employee Onboarding (Sequence Diagram)

```

sequenceDiagram
    participant NewEmployee as Emp
    participant HR
    participant IT
    Emp->>HR: Submit documents
    HR->>IT: Provision laptop
    IT-->>Emp: Setup complete
    HR-->>Emp: Send welcome kit

```

Description: A new employee submits paperwork, HR processes it, and IT provisions equipment. Note the “participant” declarations and arrow messages (**->>** solid lines with arrowheads) showing requests and responses.

Incident Handling (State Diagram)

```

stateDiagram-v2
    [*] --> Open
    Open --> Assigned : assign
    Assigned --> InProgress : startWork
    InProgress --> Resolved : fix
    Resolved --> [*] : close
    InProgress --> Escalated : needsHelp
    Escalated --> InProgress : backFromEscalation

```

Description: States of an incident ticket. The **[*]** shows start/end. Transitions have labels (e.g. **assign**, **fix**) indicating triggers.

Approval Workflow (Flowchart)


```

flowchart TD
    Request[Employee submits request] --> Manager{Manager approves?}
    Manager -->|Yes| Finance[Forward to Finance]
    Manager -->|No| Reject[Notify rejection]
    Finance --> Accountant[Process payment]
    Accountant --> Done[Complete request]

```

Description: A simple approval flow: manager accepts or rejects an employee request. Labels “Yes/No” clarify branches. This uses a downward flow (TD).

Customer Onboarding (User Journey)

```

journey
    title Customer Onboarding
    section Pre-Arrival
        "Sign up": 5: Customer, System
        "Verify email": 3: System
    section First Week
        "Initial call": 2: Sales, Customer
        "Setup complete": 4: IT
        "Welcome email": 1: Marketing

```

Description: A user journey chart with sections (phases). Each task has a score and actors listed. This shows the customer’s steps and who is involved at each step ¹⁸ .

Task Distribution (Pie Chart)

```

pie title "Monthly Requests"
    "Support": 120
    "Sales": 80
    "IT": 30
    "HR": 10

```

Description: Pie chart of service requests by department.

Best Practices for Clarity

- **Branching Flows:** Always label decision arrows (e.g. “Yes/No”) rather than embedding text in diamonds. Keep decision logic binary if possible. If multiple conditions arise, break into multiple simple decisions or use subgraphs to separate conditions. Place connectors evenly (minimize crossings) to avoid “spaghetti flow” (a tangled web of lines) ²⁶ .

- **Swimlane Simulation:** Since Mermaid lacks true lanes, simulate them with subgraphs. For example, label each subgraph by department or role. Keep minimal cross-links between subgraphs to maintain clear lanes. If you find yourself forcing many cross-lane arrows, consider whether separate charts or a sequence diagram might be clearer. (Mermaid issue trackers note swimlane support is forthcoming ²⁷, so watch for future updates.)
- **Consistent Labeling:** Use a consistent style for node/actor names. Prefer sentence case or title case, and avoid overly long labels. Abbreviations should be defined in legends or notes. For decision nodes, use clear predicates (e.g. `{Approved?}` rather than `{Yes/No}`). Label connectors rather than embedding text in shapes.
- **Text Alignment:** Node text is centered by default. If a label is long, break lines manually (use `<br>` or a newline inside quotes) to keep node shapes compact. Align bullet lists or notes left for readability. Avoid having some labels top-aligned and others centered in the same diagram.
- **Comments:** Use `%% this is a comment` lines to explain parts of the code. These are ignored by the renderer ²¹. This keeps the code maintainable.
- **Versioning:** When embedding diagrams in docs (GitHub, Notion, etc.), be aware that each platform may use a different Mermaid version. Complex or new syntax (like very recent features) may not render everywhere. Test on target platforms.

WARNING: Don't create extremely deep nested subgraphs or loops – it can be confusing. If a process is very complex, it may be better modeled as multiple simpler diagrams or using another diagram type (e.g. state or sequence).

Platform Compatibility

Mermaid is supported in many documentation and note-taking tools. Notably, **GitHub** now renders Mermaid code blocks directly in Markdown files ²⁴. **GitLab** also natively supports Mermaid. Many editors (VS Code with a Mermaid preview extension, [Typora](#), etc.) will show diagrams live as you write.

Popular platforms with native or plugin support include Notion and Obsidian (both have built-in Mermaid support) ²⁵, Atlassian Confluence/Jira (via plugins), GitBook, Joplin, and others. If your tool doesn't auto-render, you can use the [Mermaid CLI](#) or GitHub Actions to convert `.mmd` into images or SVGs for embedding. Always check that your platform's Mermaid version supports the syntax you use.

Appendix: Syntax Reference Patterns

- **General Declaration:**

```
flowchart LR
sequenceDiagram
stateDiagram-v2
classDiagram
```

```
gantt
journey
pie
```

• **Flowchart (Graph):**

```
flowchart LR
    A[Start] --> B{Decision}
    B -->|Yes| C[Outcome 1]
    B -->|No| D[Outcome 2]
```

• **Subgraph:**

```
subgraph GroupName
    Node1 --> Node2
end
```

• **Sequence Diagram:**

```
sequenceDiagram
    participant A
    participant B
    A->>B: Message
    loop Repeat
        B-->>A: Response
    end
```

• **State Diagram:**

```
stateDiagram-v2
    [*] --> State1
    State1 --> State2: event
    State2 --> [*]
```

• **Class Diagram:**

```
classDiagram
    ClassA <|-- ClassB
    ClassA --o ClassC
    ClassA : +attr
    ClassA : +method()
```

• **Gantt Chart:**

```

gantt
  title Title
  dateFormat YYYY-MM-DD
  axisFormat %Y-%m-%d
  section Name
    Task :id, 2025-01-01, 10d
    Task2: after id, 5d

```

• **User Journey:**

```

journey
  title Title
  section Phase
    "Task description": 5: Actor1, Actor2

```

• **Pie Chart:**

```

pie showData title "Title"
  "Label1": 10
  "Label2": 20.5

```

Common Anti-Patterns

Anti-pattern	Description / Remedy
Spaghetti Flow	Many crossing edges and tangled paths make the diagram unreadable. Avoid by simplifying the flow or splitting into subgraphs. (See figure below.)
Over-Nesting	Excessive subgraph nesting or multi-level loops that obscure the main logic. Flatten the structure; break the diagram into parts if needed.
Unlabeled Branches	Decision arrows without labels (e.g. leaving a fork unlabeled) confuse readers. Always label edges with conditions.
Overloaded Nodes	Nodes containing too much text or combining multiple concepts. Split them into separate steps or add notes.

Figure: An example of a tangled “spaghetti” flow (highlighted) – a common anti-pattern. Such crossed arrows and convoluted paths reduce readability.

Each anti-pattern can often be resolved by refactoring: grouping related steps, labeling clearly, or dividing the process into smaller diagrams. Remember, the goal is clarity: if you struggle to follow the Mermaid code yourself, others will too.

1 2 3 4 5 7 8 9 21 **Flowcharts Syntax | Mermaid**

<https://mermaid.js.org/syntax/flowchart.html>

6 10 11 **Sequence diagrams | Mermaid**

<https://mermaid.js.org/syntax/sequenceDiagram.html>

12 13 **State diagrams | Mermaid**

<https://mermaid.js.org/syntax/stateDiagram.html>

14 15 **Class diagrams | Mermaid**

<https://mermaid.js.org/syntax/classDiagram.html>

16 17 **Gantt diagrams | Mermaid**

<https://mermaid.js.org/syntax/gantt.html>

18 **User Journey Diagram | Mermaid**

<https://mermaid.js.org/syntax/userJourney.html>

19 **Pie chart diagrams | Mermaid**

<https://mermaid.js.org/syntax/pie.html>

20 **Diagram Syntax | Mermaid**

<https://mermaid.js.org/intro/syntax-reference.html>

22 **BPMN support · Issue #2623 · mermaid-js/mermaid · GitHub**

<https://github.com/mermaid-js/mermaid/issues/2623>

23 **Mermaid vs. PlantUML: Comparing Popular Diagramming Tools for Developers | Gleek.io | Gleek**

<https://www.gleek.io/blog/mermaid-vs-plantuml>

24 25 **Integrations | Mermaid**

<https://mermaid.js.org/ecosystem/integrations-community.html>

26 **How to display flowchart with different sections and swimlanes that can point to each other in markdown - Stack Overflow**

<https://stackoverflow.com/questions/78349919/how-to-display-flowchart-with-different-sections-and-swimlanes-that-can-point-to>

27 **Use swimlanes in flowchart diagram · Issue #2028 · mermaid-js/mermaid · GitHub**

<https://github.com/mermaid-js/mermaid/issues/2028>